

Distributed Operating Systems (DOS)

Lab 2 Report: Replication, Caching, and Consistency (Bazar.com)

Rasmiya Thaher & Shahd Fakhouri

1. Overview

This lab extends the multi-tier Bazar.com system from Lab 1 by adding replication, an in-memory cache at the front-end, and consistency mechanisms. The goal is to reduce request latency under higher workload while ensuring correct behavior when the catalog is updated due to purchases or stock updates.

2. System Architecture

The system follows a microservices architecture using REST APIs between components:

- Front-end service (single instance): receives all client requests, performs load balancing, and caches read responses.
- Catalog service (two replicas): maintains the catalog (book information, stock, price).
- Order service (two replicas): processes purchases and updates stock by coordinating with the catalog.

All services are deployed as separate processes/containers and communicate over HTTP.

3. Replication and Load Balancing

Both backend services are replicated (2 replicas each). The front-end implements a Round-Robin load balancing policy to distribute incoming requests across replicas. Read requests (search/info) are forwarded to catalog replicas; write requests (purchase) are forwarded to order replicas. Round-Robin is implemented by keeping an index pointer for each service type and alternating between replica 1 and replica 2.

4. Caching and Consistency

The front-end maintains an in-memory cache for read-only catalog queries (primarily info and optionally search). Write operations are never served from cache. To keep cached data correct after any write, the backend uses server-push invalidation: before committing a write to the catalog data, the backend sends an invalidate message to the front-end cache to remove the affected item. After invalidation, the next read becomes a cache miss and is fetched from the catalog replica, then stored again.

5. How to Run

To run the system, the services should be started in the following order.

1. Start the catalog service replicas. Catalog replica 1 runs on port 5001 and catalog replica 2 runs on port 5003. Each replica is started from its own directory using the command `python app.py`.
2. Start the order service replicas. Order replica 1 runs on port 5002 and order replica 2 runs on port 5004. Each replica is started using the command `python app.py`.
3. Start the front-end service. The front-end server runs on port 5000 and is started using the command `python app.py`.

Clients send requests to the front-end server, which performs load balancing and caching and forwards requests to the appropriate backend replicas.

6. Output and Experiments

Catalog Replica 1 running and ready to receive REST requests (search/info/update).

```
PS C:\Users\shahd\OneDrive\Desktop\DOS\catalog> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.1.105:5001
Press CTRL+C to quit
```

Catalog Replica 2 running. It should have the same dataset and respond correctly to the same requests.

```
PS C:\Users\shahd\OneDrive\Desktop\DOS\catalog> $env:PORT="5003"
PS C:\Users\shahd\OneDrive\Desktop\DOS\catalog> $env:FRONTEND_URL="http://localhost:5000"
PS C:\Users\shahd\OneDrive\Desktop\DOS\catalog> $env:CATALOG_PEERS="http://localhost:5001"
PS C:\Users\shahd\OneDrive\Desktop\DOS\catalog> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://192.168.1.105:5003
Press CTRL+C to quit
```

Front-end service running. All client requests are sent to the front-end. It performs load balancing and caching.

```
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.105:5000
Press CTRL+C to quit
```

Order Replica 1 running. It handles purchase requests and coordinates with the catalog to decrement stock.

```
PS C:\Users\shahd\OneDrive\Desktop\DOS> cd "C:\Users\shahd\OneDrive\Desktop\DOS\order\"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> $env:PORT="5002"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> $env:CATALOG_REPLICAS="http://localhost:5001,http://local
host:5003"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> $env:ORDER_PEERS="http://localhost:5004"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.1.105:5002
Press CTRL+C to quit
```

Order Replica 2 running. It should stay consistent with Order Replica 1 (replicated order log/state).

```
PS C:\Users\shahd\OneDrive\Desktop\DOS> cd "C:\Users\shahd\OneDrive\Desktop\DOS\order\"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> $env:PORT="5004"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> $env:CATALOG_REPLICAS="http://localhost:5001,http://local
host:5003"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> $env:ORDER_PEERS="http://localhost:5002"
PS C:\Users\shahd\OneDrive\Desktop\DOS\order> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5004
* Running on http://192.168.1.105:5004
Press CTRL+C to quit
```

Round Robin Load Balancing Test

These screenshots demonstrate the round-robin load balancing mechanism implemented in the front end server. Repeated requests to the same endpoint (/info/2) are sent alternately to different catalog replicas.

In the responses, the source field shows that the requests are handled by different catalog replicas (http://localhost:5001 and http://localhost:5003) in a rotating manner. This confirms that the front-end server distributes incoming requests evenly across the available catalog replicas using a round-robin policy.

GET

http://localhost:5000/info/2

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (5)

Test Results (1/1)

200 OK · 2.

{ } JSON

Preview

Visualize

```
1 {
2   "data": {
3     "price": 50.0,
4     "quantity": 5,
5     "title": "RPCs for Noobs"
6   },
7   "source": "http://localhost:5001"
8 }
```

GET

http://localhost:5000/info/2

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (5)

Test Results (1/1)

200 OK · 2.02 s

{ } JSON

Preview

Visualize

```
1 {
2   "data": {
3     "price": 50.0,
4     "quantity": 5,
5     "title": "RPCs for Noobs"
6   },
7   "source": "http://localhost:5003"
8 }
```

GET

http://localhost:5000/info/2

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Descrip
	Key	Value	Descrip

Body

Cookies

Headers (5)

Test Results (1/1)

200 OK

{ } JSON

Preview

Visualize

```
1 {
2   "data": {
3     "price": 50.0,
4     "quantity": 5,
5     "title": "RPCs for Noobs"
6   },
7   "source": "http://localhost:5001"
8 }
```

GET http://localhost:5000/info/2

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results (1/1) 200 OK

{ } JSON Preview Visualize

```
1 {
2   "data": {
3     "price": 50.0,
4     "quantity": 5,
5     "title": "RPCs for Noobs"
6   },
7   "source": "http://localhost:5003"
8 }
```

cache test

These screenshots show that the first request results in a cache miss and is served by a catalog replica, while the second request is served directly from the cache, as indicated by the source field.

GET http://localhost:5000/info/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results (1/1) 200 OK • 2.04 s • 299 B

{ } JSON Preview Visualize

```
1 {
2   "data": {
3     "price": 123.0,
4     "quantity": 10,
5     "title": "How to get a good grade in DOS in 40 minutes a day"
6   },
7   "source": "http://localhost:5001"
8 }
```

GET http://localhost:5000/info/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results (1/1) 200 OK 5 ms

{ } JSON Preview Visualize

```
1 {
2   "data": {
3     "price": 123.0,
4     "quantity": 10,
5     "title": "How to get a good grade in DOS in 40 minutes a day"
6   },
7   "source": "cache"
8 }
```

Cache Status

This step is used to check the internal state of the front-end cache. It confirms that caching is enabled and shows the cached entries currently stored before performing any write operation.

GET http://localhost:5000/cache-status

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Descriptio
Key	Value	Descriptio

Body Cookies Headers (5) Test Results (1/1) 200 OK

{ } JSON Preview Visualize

```
1 {
2   "capacity": 50,
3   "enabled": true,
4   "keys": [
5     "info:2",
6     "search:distributed systems"
7   ],
8   "size": 2
9 }
```

Purchase (WRITE)

Here, a purchase request is issued, which represents a write operation. This request is intentionally not served from the cache and is forwarded to the order service, which updates the catalog by decreasing the item quantity.

POST <http://localhost:5000/purchase/1>

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results (1/1) 200 OK • 7.74 s

{ } JSON Preview Visualize

```

1  {
2    "data": {
3      "message": "bought book How to get a good grade in DOS in 40 minutes a day",
4      "order": {
5        "item_id": 1,
6        "order_id": 22,
7        "price": 123.0,
8        "timestamp": "2025-12-21T21:21:48.640037Z",
9        "title": "How to get a good grade in DOS in 40 minutes a day"
10     },
11     "status": "success"
12   },
13   "source": "http://localhost:5002"
14 }

```

Info again (must NOT be from cache now)

After the purchase, the same information request is repeated. The response is fetched from a catalog replica instead of the cache, showing that the cached entry was invalidated correctly after the write operation.

GET <http://localhost:5000/info/1>

Docs Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results (1/1) 200 OK • 2.08 s • 298

{ } JSON Preview Visualize

```

1  {
2    "data": {
3      "price": 123.0,
4      "quantity": 9,
5      "title": "How to get a good grade in DOS in 40 minutes a day"
6    },
7    "source": "http://localhost:5001"
8  }

```


Write update to Catalog replica 1

In this step, an explicit update is applied to catalog replica 1. The catalog data is modified locally, and the update is then propagated to the other catalog replica to keep both replicas consistent.

The screenshot shows a REST client interface with a POST request to `http://localhost:5001/update`. The request body is a JSON object: `{ "item_id": 1, "price": 55 }`. The response is a `200 OK` status with a JSON body: `{ "status": "ok" }`.

```
1 {
2   "item_id": 1,
3   "price": 55
4 }
5
```

```
1 {
2   "status": "ok"
3 }
```

Read from Catalog replica 2

This request is sent to catalog replica 2 to verify that the update applied on the first replica has been successfully replicated. The correct response confirms that catalog replication is working as intended.

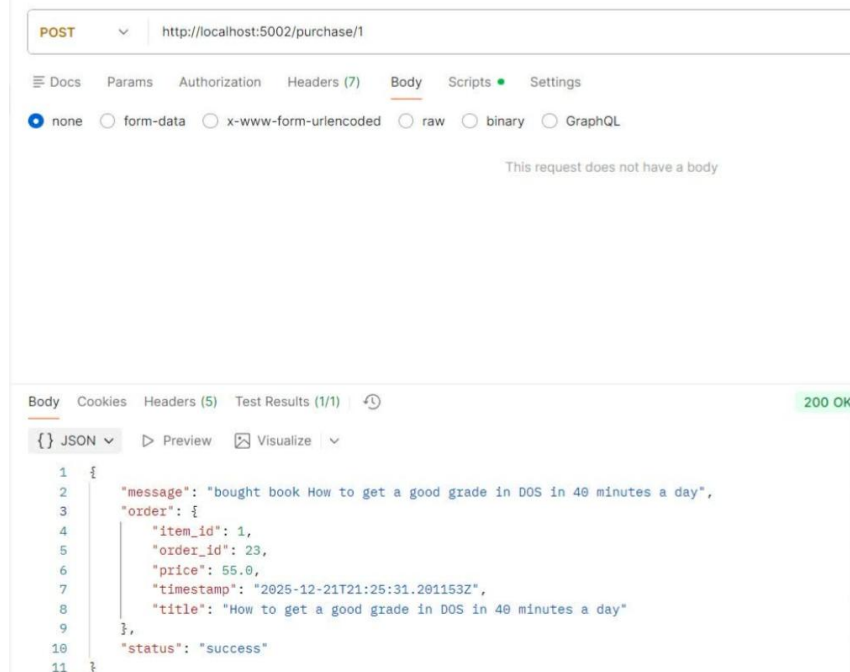
The screenshot shows a REST client interface with a GET request to `http://localhost:5003/info/1`. The response is a `200 OK` status with a JSON body: `{ "item_id": 1, "price": 55.0, "quantity": 9, "title": "How to get a good grade in DOS in 40 minutes a day" }`.

```
1 {
2   "item_id": 1,
3   "price": 55
4 }
5
```

```
1 {
2   "price": 55.0,
3   "quantity": 9,
4   "title": "How to get a good grade in DOS in 40 minutes a day"
5 }
```

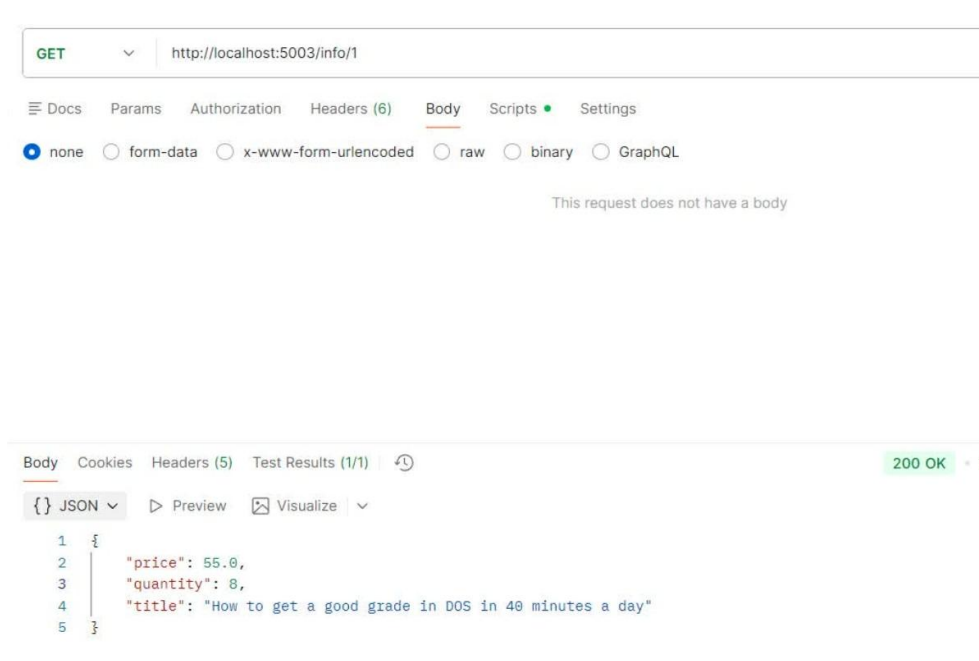

Purchase directly from order replica 1

A purchase request is sent directly to order replica 1. The order service validates item availability through the catalog, performs the update, and logs the order before forwarding it to its peer replica.



Verify data updated through catalog replica 2

Finally, the item information is retrieved through catalog replica 2 to ensure that all previous write operations are reflected consistently across replicas.



7. Performance Evaluation

We measure average response time for read (query/info/search) and write (purchase) operations, comparing runs with caching disabled vs enabled. For each case, we issue the same set of requests multiple times and compute the average latency in milliseconds.

Mode	Avg info/search latency (ms)	Avg purchase latency (ms)
Without cache	2070 ms	6400 ms
With cache	26 ms	6450 ms

Caching greatly reduces read (info/search) latency, lowering the average response time from 2070 ms to 26 ms by avoiding repeated requests to catalog replicas. However, purchase latency remains almost the same (about 6.4 seconds) with or without caching, since purchase operations involve synchronous inter-service communication and do not benefit from cached data.

Cache Consistency Overhead

To measure invalidation overhead, we first warm the cache by issuing an info request for an item twice (hit expected on second request). Then we issue a purchase (write), which triggers invalidation. Immediately after, we issue info again (cache miss expected) and record the latency. Report: (1) invalidate time/overhead (if logged), and (2) latency of the first read after invalidation (miss).

Metric	Value (ms)	Notes
Latency after invalidation (cache miss)	2070 ms	Expected miss; fetched from catalog replica

After a purchase operation, the cached item is invalidated, causing the next read to result in a cache miss and be served from a catalog replica, which explains the higher latency.

8. Design Tradeoffs and Improvements

Caching reduces latency for repeated read requests but introduces invalidation overhead on writes. Replication improves throughput and availability but requires synchronization between replicas to maintain consistency. Possible improvements include adding an eviction policy (LRU) with a cache size limit, implementing least-loaded load balancing, and adding health checks/failover for replica outages.